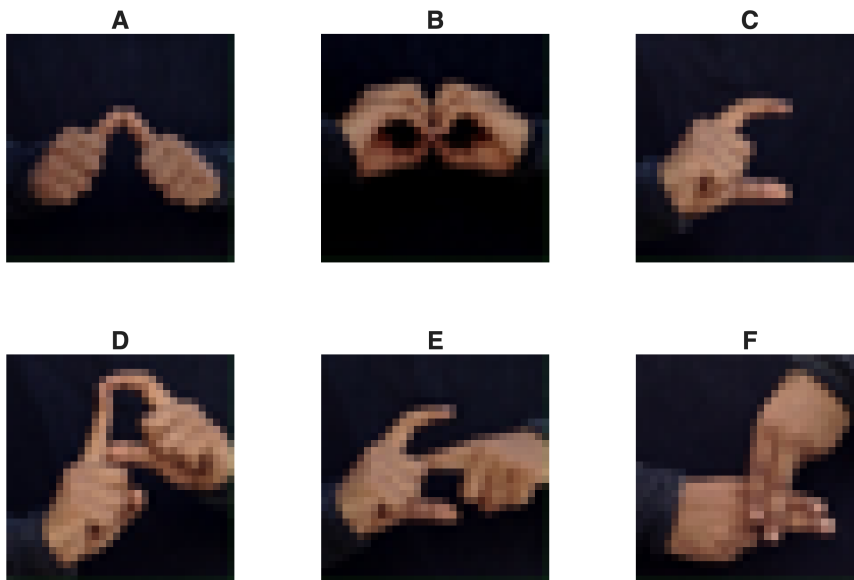


```

% loading my path
path = '/Users/melisakocak/Downloads/data';
[data, classes] = load_helper(path, 800);
normalized = data/255;
% 800 samples 26 classes--> data is reduced to about 20,000 After loading
the data, visualizing the first 6 letters
letters = ["A", "B", "C", "D", "E", "F"];
figure;
% first 6 letters
for i = 1:6
    % plotting 2x3
    subplot(2, 3, i)
    % taking first sample
    first_image = find(classes == letters(i), 1);
    % using imshow to display
    imshow(reshape(normalized(first_image,:), [32 32 3]));
    title(letters(i));
end

```



Here are the helper functions in order to load my image dataset.

First helper is to get only letter directories, this way I reduced my dataset and speed up the execution time.

```

function images = get_letter_directories(path)
    tmp = dir(path);
    directory = tmp([tmp.isdir]);
    % only working with the alphabet

```

```

letters =
["A","B","C","D","E","F","G","H","I","J","K","L","M","N","O","P","Q","R","S",
,"T","U","V","W","X","Y","Z"];
letter_names = cellstr(letters);
dir_names = {directory.name};
% letter directories check
check_mem = ismember(dir_names, letter_names);
images = directory(check_mem);
end

```

Now, the main helper to load the images:

```

function [data, classes] = load_helper(path, num)
directories = get_letter_directories(path);
% 800
images_length = length(directories) * num;
% reducing dimension to speed
reduced_dim = [32 32];
% instead of 128 x 128 x 3
total_reduced_dim = (prod(reduced_dim)) * 3;
% preallocating
data = zeros(images_length, total_reduced_dim);
classes = strings(images_length, 1);
% while loop
i = 1;
% letter folder
% 26 letters in alphabet
while i <= 26
    dir_names = directories(i).name;
    % JPEG FORMAT
    jpeg_images = dir(fullfile(path, dir_names, '*.jpg'));
    % image inside but for each sample
    for j = 1:num
        image_names = jpeg_images(j).name;
        folders = jpeg_images(j).folder;
        % reading then resize
        tmp = imread(fullfile(folders, image_names));
        image_classes = imresize(tmp, reduced_dim);
        rows_filled = (i - 1) * num;
        curr = rows_filled + j;
        flattened = image_classes(:);
        % reassigning
        data(curr, :) = double(flattened');
        classes(curr) = dir_names;
    end
    i = i + 1;
end
end
end

```

Initialization of split: 80 for training, 10 for testing, 10 for validation

```
% check
disp(size(data, 1));
```

```
20800
```

```
images_length = size(data, 1);
% to have same result, from sample code
rng('default');
p = randperm(images_length);
% 80 10 10
% manual split
train_percent_length = .8 * images_length;
% 80+ 10 = 90
validation_percent_length = .9 * images_length;
% initializations used for svm
% training
training_elements = p(1:train_percent_length);
normalized_tra = normalized(training_elements, :);
% check
disp(size(normalized_tra));
```

```
16640
```

```
3072
```

```
training_labels = classes(training_elements,:);
% validation
validation_elements = p(train_percent_length+1:validation_percent_length);
normalized_ve = normalized(validation_elements, :);
% check
disp(size(normalized_ve));
```

```
2080
```

```
3072
```

```
validation_labels = classes(validation_elements,:);
% testing
testing_element = p(validation_percent_length+1:end);
normalized_te = normalized(testing_element, :);
% check
disp(size(normalized_te));
```

```
2080
```

```
3072
```

```
testing_labels = classes(testing_element,:);
% making sure its 26 and no duplicates
class_names = unique(classes);
```

Kernel SVM for classification. This documentation was used as a reference: <https://www.mathworks.com/help/stats/templatesvm.html>

```
function svm_helper(training_d, training_c, validation_d, validation_c,
testing_d, testing_c, method)
    % type and classification
    t = templateSVM('KernelFunction','gaussian');
```

```

% only training one
MdlSV = fitcecoc(training_d, training_c, 'Learners', t);
% finding posterior with fitcecoc
% like sample code
% ~ for fixing warnings
[labels, ~] = predict(MdlSV, validation_d);
% sample code
[testing_labels, t_posts] = predict(MdlSV, testing_d);
disp('The first five class predictions with their probabilities:');
disp(testing_labels(1:5));
disp(t_posts(1:5, :));
% c is a param
% true for cm
% confusion matrix gets printed in helper
[validation_a, ~, validation_f1] =
measures_of_success_helper(validation_c, labels, [method 'Validation'],
true);
[testing_a, ~, testing_f1] = measures_of_success_helper(testing_c,
testing_labels, [method 'Testing'], true);
fprintf('F1 score for validation dataset is %f\n', validation_f1);
fprintf('F1 score for testing dataset is %f\n', testing_f1);
fprintf('Accuracy for validation dataset is %f\n', validation_a);
fprintf('Accuracy for testing dataset is %f', testing_a);
end

```

Cross validation helper was also applied to each representation. This is to improve and debug accuracy and f1. It is running 3 different SVM models. For reference, I used the documentation: <https://www.mathworks.com/help/stats/classreg.learning.partition.classificationpartitionedmodel.kfoldloss.html>

```

function cross_val_helper(d,c, method)
ksvm = templateSVM('KernelFunction','gaussian');
CVMdl = fitcecoc(d, c, 'Learners', ksvm, 'KFold', 3);
% referenced
L = kfoldLoss(CVMdl);
names = kfoldPredict(CVMdl);
[~, ~, f1] = measures_of_success_helper(c, names, [method '3 Fold Cross
Validation'], false);
% confusion matrix gets printed in helper
fprintf('F1 score after 3 fold cross validation is %f\n', f1);
% 1- loss gives accuracy since loss shows the missclassified
fprintf('Accuracy after 3 fold cross validation is %f', 1-L);
end

```

Metric evaluation helper for f1, accuracy and confusion matrix.

The referenced used:

- <https://www.mathworks.com/help/stats/confusionmat.html>
- <https://stackoverflow.com/questions/40729875/calculate-precision-and-recall-in-a-confusion-matrix>

```

function [a,c, f1] = measures_of_success_helper(val, pred, method, flag)
    % error fix, needs to be same type
    val = string(val);
    pred = string(pred);
    total = length(pred);
    numerator = sum(val == pred);
    % to see the clear formula
    a = numerator / total;
    % confusion matrix after each method except cross val
    cm = confusionmat(val, pred);
    if flag
        figure;
        c = confusionchart(val, pred);
        c.Title = (['Confusion matrix for ' method]);
    else
        c = [];
        % empty
    end
    % getting vals from chart for f1
    true_positive = diag(cm);
    false_positive = sum(cm,1)' - true_positive;
    false_negative = sum(cm,2) - true_positive;
    % f1 = 2 times precision x recall over precision + recall
    tp_fp = true_positive + false_positive;
    tp_fn = true_positive + false_negative;
    precision = true_positive ./ tp_fp;
    recall = true_positive ./ tp_fn;
    score = 2 .* (precision.*recall) ./ (precision+recall);
    f1 = sum(score)/ length(score);
end

% helper to reduce redundancy
function point = row_helper(l, d)
    point = (l == d);
end

```

Raw Data Representation

```

% sanity check
info = tabulate(classes);
disp(info);

```

```

{'A'}    {[800]}    {[3.8462]}
{'B'}    {[800]}    {[3.8462]}
{'C'}    {[800]}    {[3.8462]}
{'D'}    {[800]}    {[3.8462]}
{'E'}    {[800]}    {[3.8462]}
{'F'}    {[800]}    {[3.8462]}
{'G'}    {[800]}    {[3.8462]}
{'H'}    {[800]}    {[3.8462]}
{'I'}    {[800]}    {[3.8462]}
{'J'}    {[800]}    {[3.8462]}

```

```

{'K'}    {[800]}    {[3.8462]}
{'L'}    {[800]}    {[3.8462]}
{'M'}    {[800]}    {[3.8462]}
{'N'}    {[800]}    {[3.8462]}
{'O'}    {[800]}    {[3.8462]}
{'P'}    {[800]}    {[3.8462]}
{'Q'}    {[800]}    {[3.8462]}
{'R'}    {[800]}    {[3.8462]}
{'S'}    {[800]}    {[3.8462]}
{'T'}    {[800]}    {[3.8462]}
{'U'}    {[800]}    {[3.8462]}
{'V'}    {[800]}    {[3.8462]}
{'W'}    {[800]}    {[3.8462]}
{'X'}    {[800]}    {[3.8462]}
{'Y'}    {[800]}    {[3.8462]}
{'Z'}    {[800]}    {[3.8462]}

```

```

% plotting trainign and validation and testing
t1s = tic;
% for scatter
dimension = size(normalized,2);
random_projection = randn(dimension, 3);
% to project onto 3D with the split
train_matrix = normalized_tra * random_projection;
validation_matrix = normalized_ve * random_projection;
testing_matrix = normalized_te * random_projection;
t1 = toc(t1s);
fprintf('Raw Data took %f seconds.', t1);

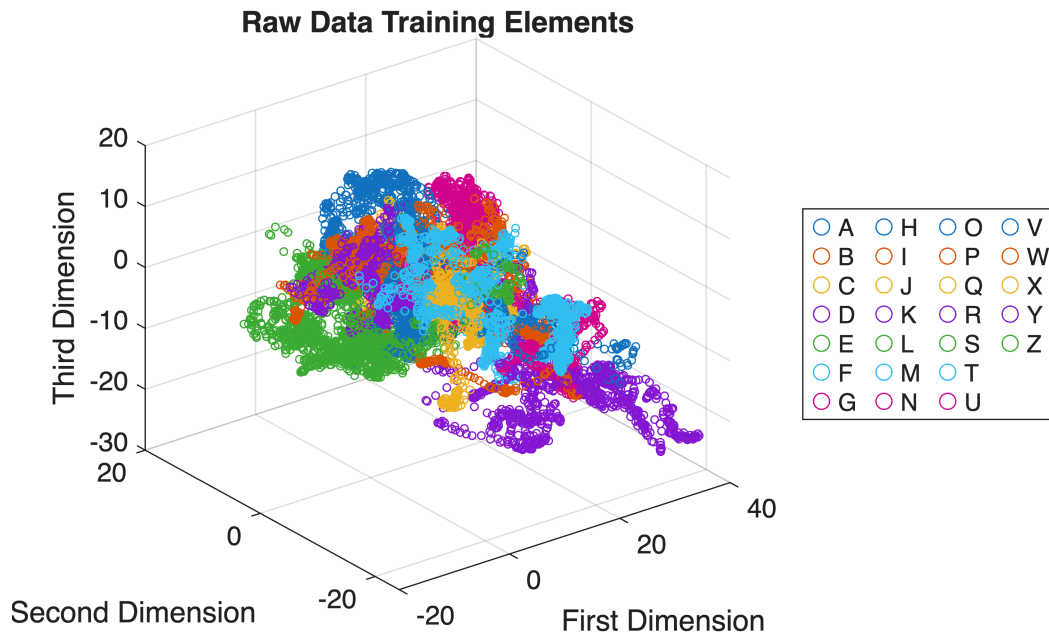
```

Raw Data took 0.049266 seconds.

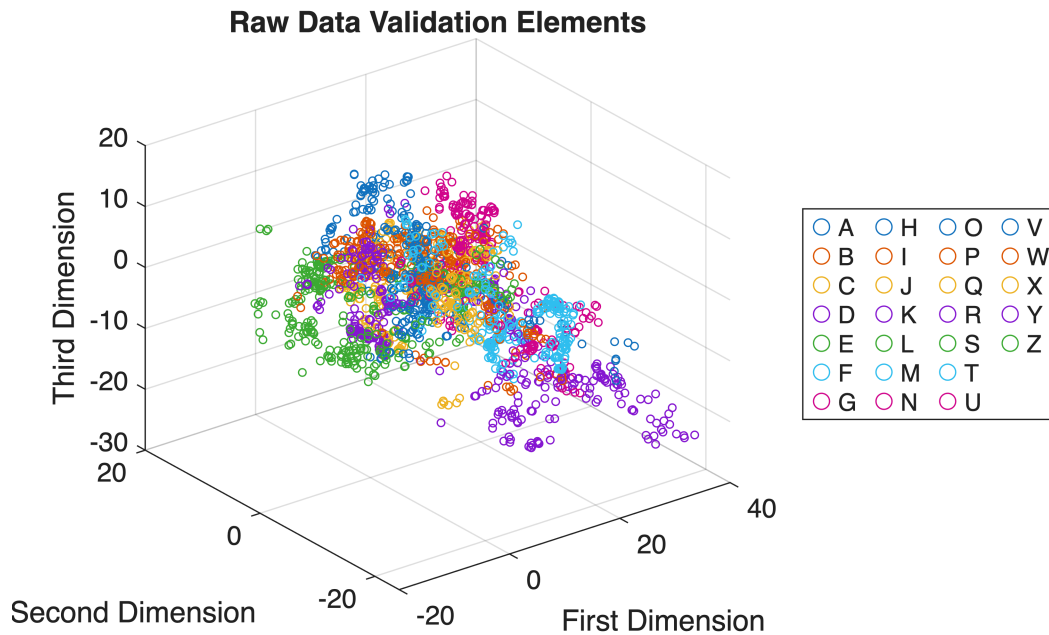
```

figure;
hold on;
for i = 1:26
    point = row_helper(classes, class_names(i));
    t_p = point(training_elements);
    scatter3(train_matrix(t_p, 1), train_matrix(t_p, 2), train_matrix(t_p,
3), 8);
end
title('Raw Data Training Elements');
grid on;
xlabel('First Dimension');
ylabel('Second Dimension');
zlabel('Third Dimension');
view(3);
table = legend(cellstr(class_names), 'Location','eastoutside');
table.NumColumns = 4;
cmap = jet(10);
colormap(cmap);
drawnow;
% shorter labels overall so they don't overlap
hold off;

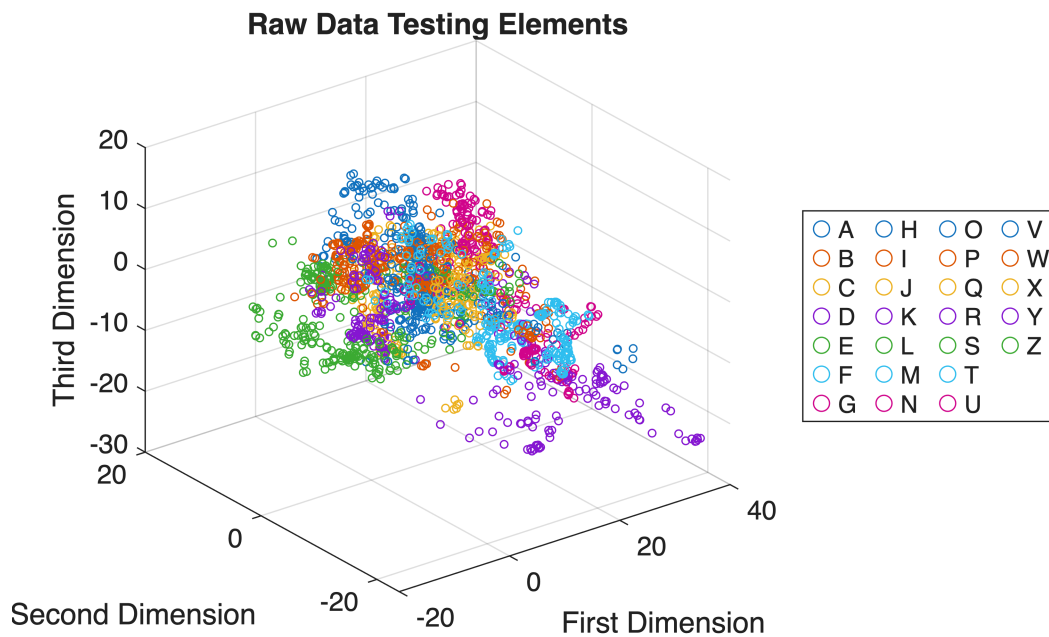
```



```
figure;
hold on;
for i = 1:26
    point = row_helper(classes, class_names(i));
    v_p = point(validation_elements);
    scatter3(validation_matrix(v_p, 1), validation_matrix(v_p, 2),
validation_matrix(v_p, 3), 8);
end
title('Raw Data Validation Elements');
grid on;
xlabel('First Dimension');
ylabel('Second Dimension');
zlabel('Third Dimension');
view(3);
cmap = jet(10);
colormap(cmap);
drawnow;
table = legend(cellstr(class_names), 'Location','eastoutside');
table.NumColumns = 4;
hold off;
```



```
figure;
hold on;
for i = 1:26
    point = row_helper(classes, class_names(i));
    te_p = point(testing_element);
    scatter3(testing_matrix(te_p, 1), testing_matrix(te_p,
2),testing_matrix(te_p, 3), 8);
end
title('Raw Data Testing Elements');
grid on;
% making wider
xlabel('First Dimension');
ylabel('Second Dimension');
zlabel('Third Dimension');
view(3);
cmap = jet(10);
colormap(cmap);
drawnow;
table = legend(cellstr(class_names), 'Location','eastoutside');
table.NumColumns = 4;
hold off;
```



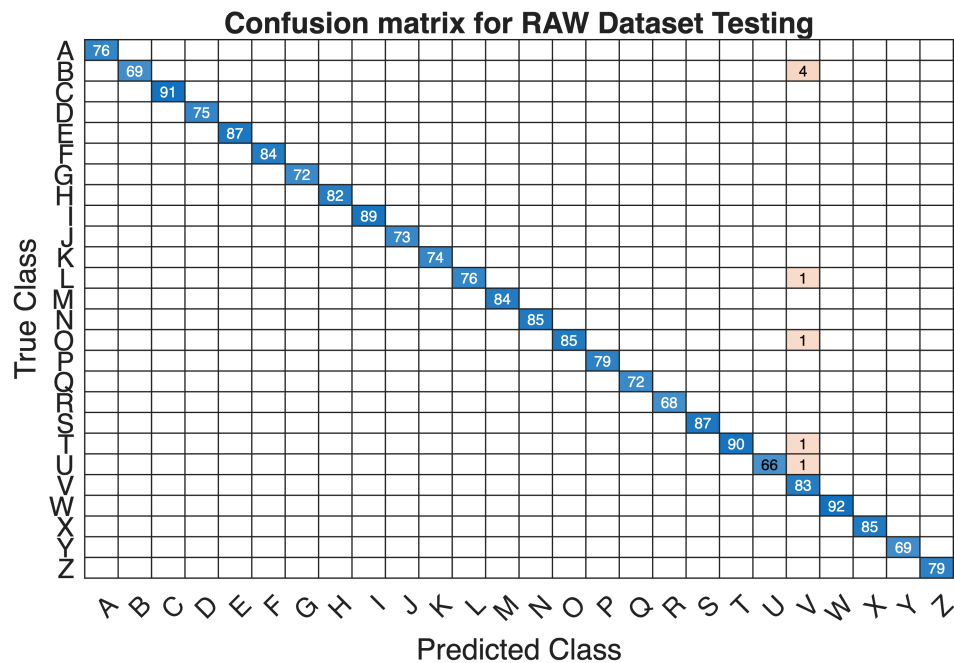
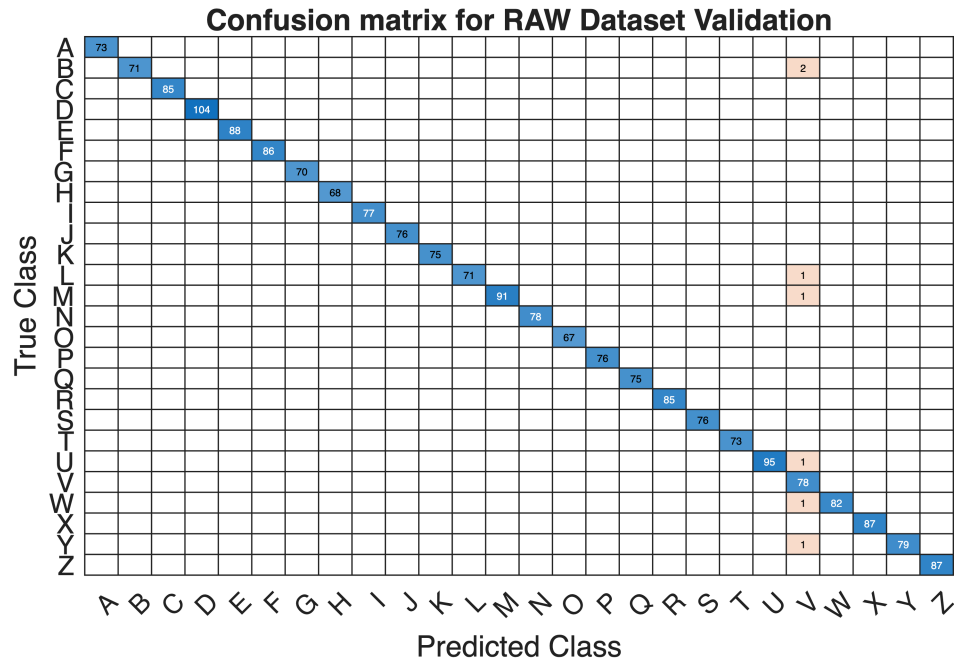
```
% calling helpers
```

```
svm_helper(normalized_tra, training_labels, normalized_ve,  
validation_labels, normalized_te, testing_labels, 'RAW Dataset');
```

The first five class predictions with their probabilities:

```
{'H'}  
{'O'}  
{'K'}  
{'B'}  
{'D'}
```

-0.4907	-0.3987	-0.3731	-0.4545	-0.6514	-0.6331	-0.6261	0	-0.5117	-0.6611	-0.6514
-0.4827	-0.3908	-0.3653	-0.4465	-0.6447	-0.6262	-0.6191	-0.5369	-0.5038	-0.6546	-0.6514
-0.4951	-0.4024	-0.3765	-0.4587	-0.6563	-0.6381	-0.6310	-0.5494	-0.5162	-0.6661	-0.6514
-0.4838	-0.0064	-0.3663	-0.4475	-0.6455	-0.6270	-0.6200	-0.5379	-0.5048	-0.6554	-0.6514
-0.4862	-0.3943	-0.3687	-0.0070	-0.6475	-0.6291	-0.6220	-0.5402	-0.5072	-0.6573	-0.6514



F1 score for validation dataset is 0.996659
 F1 score for testing dataset is 0.996170
 Accuracy for validation dataset is 0.996635
 Accuracy for testing dataset is 0.996154

```
tmp = [];
% reducing for cross val to process faster
cross_val_raw_str = strings(0,1);
for i = 1:26
    class_idx = row_helper(classes, class_names(i));
    point = find(class_idx);
    % 100 num of class
    raw_p = point(1:100);
    tmp = [tmp; normalized(raw_p,:)];
end
```

```

    cross_val_raw_str = [cross_val_raw_str; repmat(class_names(i),100, 1)];
end
cross_val_helper(tmp, cross_val_raw_str, 'RAW Dataset ');

```

F1 score after 3 fold cross validation is 0.981238
 Accuracy after 3 fold cross validation is 0.979231

PCA, using built in function and cumsum documentation: <https://www.mathworks.com/help/stats/pca.html>

```

% need it to use the normal pca
rmpath('/Users/melisakocak/Documents/MATLAB/drtoolbox/techniques');
rehash;
% only using training
t2s = tic;
[pc_t, basis_t, ~, ~, explained_t, mu] = pca(normalized_tra);
training_pca = basis_t(:, 1:50);
val_dif = normalized_ve - mu;
testing_dif = normalized_te - mu;
fifty = pc_t(:, 1:50);
validation_pca = val_dif * fifty;
testing_pca = testing_dif * fifty;
t2 = toc(t2s);
fprintf('PCA took %f seconds.', t2);

```

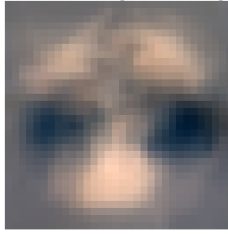
PCA took 5.511085 seconds.

```

% raw
pixels = double(data);
[pc_r, basis_r, ~, ~, explained_r] = pca(pixels);
% normalized
pixels_normalized = double(normalized);
[pc_n, basis_n, ~, ~, explained_n] = pca(pixels_normalized);
figure;
subplot(2,1,1);
% using mat2gray so it is more visible
imshow(mat2gray(reshape(pc_r(:,1), [32 32 3])));
title("The First Principal Component");
subplot(2,1,2);
imshow(mat2gray(reshape(pc_r(:,end), [32 32 3])));
title("The Last Principal Component");

```

The First Principal Component

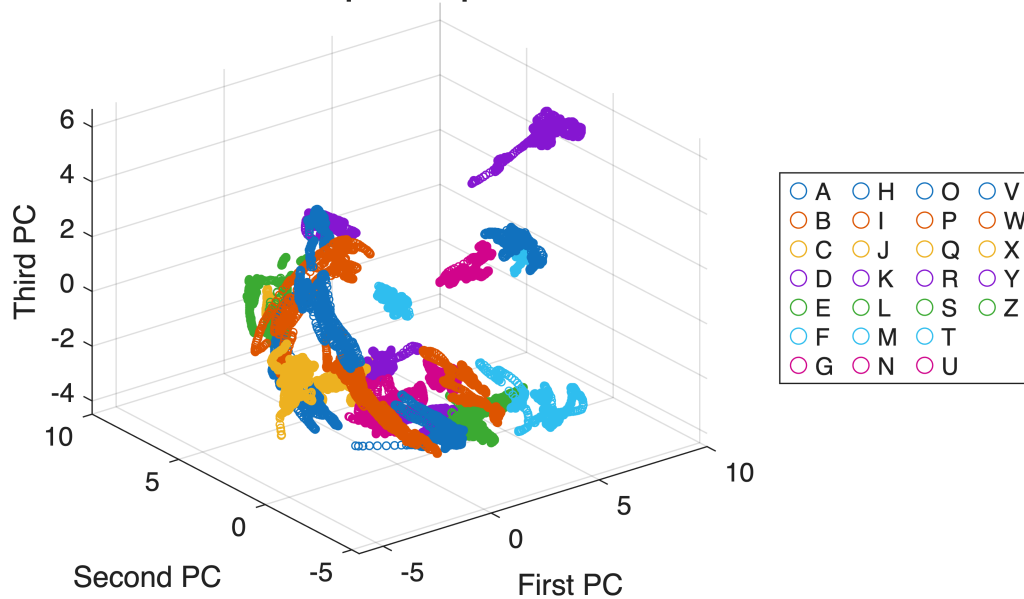


The Last Principal Component



```
% first three coefficients
coefficient_basis = basis_n(:, 1:3);
figure;
hold on;
for i = 1:26
    point = row_helper(classes, class_names(i));
    scatter3(coefficient_basis(point,1), coefficient_basis(point,2),
coefficient_basis(point,3), 8);
end
xlabel('First PC');
ylabel('Second PC');
zlabel('Third PC');
% moving table to the side, so more points are visible
table = legend(cellstr(class_names), 'Location', 'eastoutside');
table.NumColumns = 4;
cmap = jet(10);
colormap(cmap);
drawnow;
title('3D Scatter Plot of 3 Principal Components in the new Basis');
grid on;
view(3);
hold off;
```

3D Scatter Plot of 3 Principal Components in the new Basis

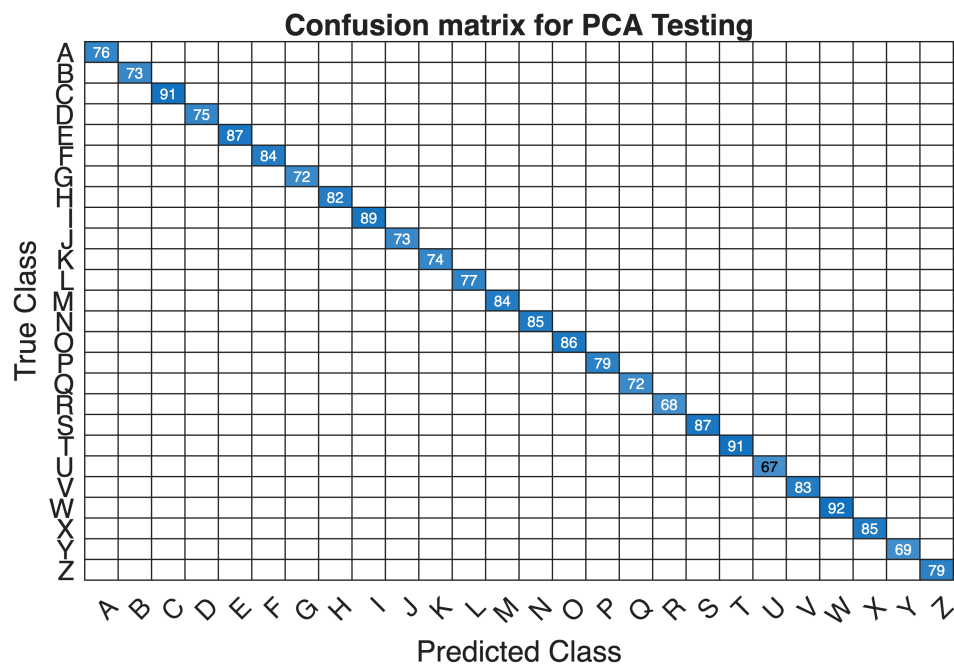
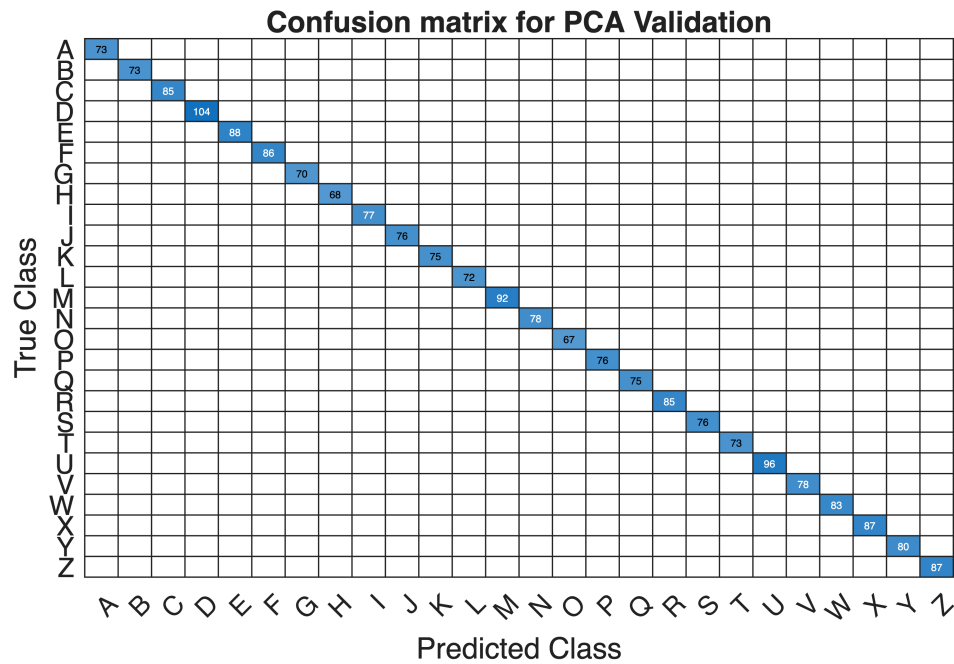


```
% for svm comparison in the analysis using normalized
svm_helper(training_pca, training_labels, validation_pca,
validation_labels, testing_pca, testing_labels, 'PCA ');
```

The first five class predictions with their probabilities:

```
{'H'}
{'O'}
{'K'}
{'B'}
{'D'}
```

-0.5002	-0.3456	-0.3479	-0.4187	-0.6181	-0.6549	-0.6730	0	-0.5702	-0.6226	-0.6177
-0.4902	-0.3353	-0.3376	-0.4083	-0.6095	-0.6470	-0.6654	-0.5141	-0.5610	-0.6141	-0.6177
-0.5073	-0.3503	-0.3526	-0.4247	-0.6260	-0.6629	-0.6809	-0.5312	-0.5779	-0.6305	-0.6177
-0.4916	0	-0.3392	-0.4098	-0.6106	-0.6480	-0.6663	-0.5155	-0.5622	-0.6152	-0.6177
-0.4947	-0.3401	-0.3424	-0.0004	-0.6132	-0.6503	-0.6686	-0.5185	-0.5650	-0.6177	-0.6177



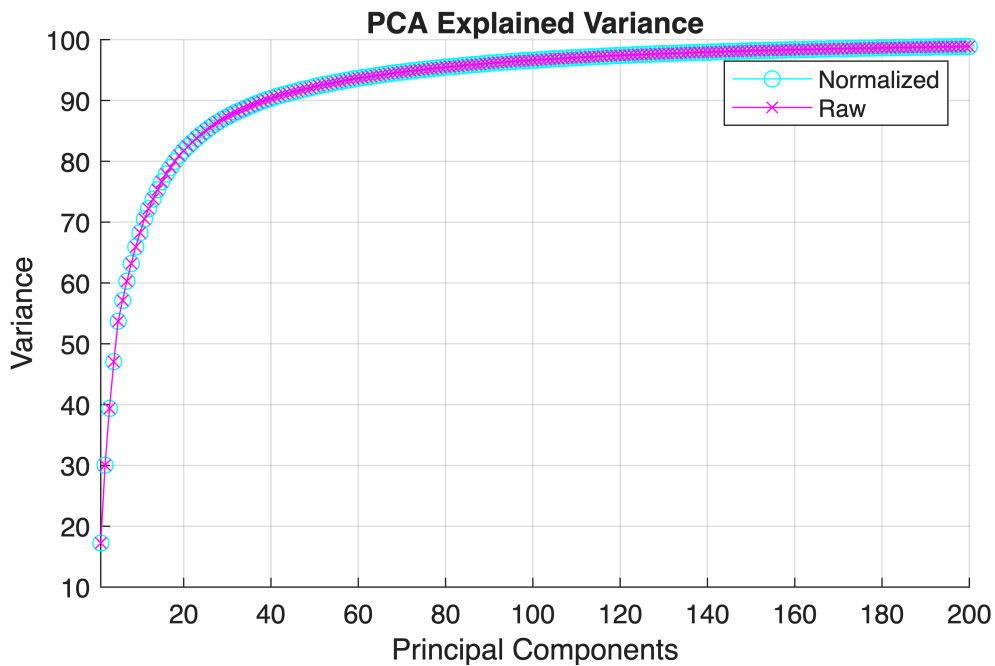
F1 score for validation dataset is 1.000000
 F1 score for testing dataset is 1.000000
 Accuracy for validation dataset is 1.000000
 Accuracy for testing dataset is 1.000000

```
cross_val_helper(training_pca,training_labels, 'PCA ');
```

F1 score after 3 fold cross validation is 0.999464
 Accuracy after 3 fold cross validation is 0.999459

```
figure;  
% plotting both in case there is a difference  
hold on;  
explained_plot_n = cumsum(explained_n);  
explained_plot = cumsum(explained_r);
```

```
% forming marker and diff color
plot(explained_plot_n, '-oc');
plot(explained_plot, '-xm');
xlim([1 200]);
xlabel('Principal Components');
ylabel('Variance');
legend('Normalized', 'Raw');
grid on;
title('PCA Explained Variance');
hold off;
```



```
fprintf('95% raw normalized variability explained by %d principal components.', find(explained_plot_n>=95,1))
```

95% raw normalized variability explained by 74 principal components.

```
fprintf('95% raw variability explained by %d principal components.', find(explained_plot>=95,1))
```

95% raw variability explained by 74 principal components.

```
% adding it back so dr tool box can be used for the rest of the methods
addpath('/Users/melisakocak/Documents/MATLAB/drtoolbox/techniques')
rehash
```

Now, Kernel PCA

```
% like sample codes
trainset = [];
trainset_norm = [];
labels = [];
t3s = tic;
```

```

for i = 1:26
    point = find(row_helper(classes, class_names(i)));
    % num per class is 100
    point= point(1:100);
    trainset =[trainset; data(point,:)];
    trainset_norm =[trainset_norm; normalized(point,:)];
    labels = [labels; repmat(class_names(i),100, 1)];
    % so its remained as letters
end
% using more dim to get higher acc, f1, better cm
[reduced, ~] = compute_mapping(trainset, 'KernelPCA', 40);

```

Computing kernel matrix...
 Eigenanalysis of kernel matrix...
 Computing final embedding...

```
[reduced_norm, ~] = compute_mapping(trainset_norm, 'KernelPCA', 40);
```

Computing kernel matrix...
 Eigenanalysis of kernel matrix...
 Computing final embedding...

```

t3 = toc(t3s);
fprintf('Kernel PCA took %f seconds', t3);

```

Kernel PCA took 3.592986 seconds

```

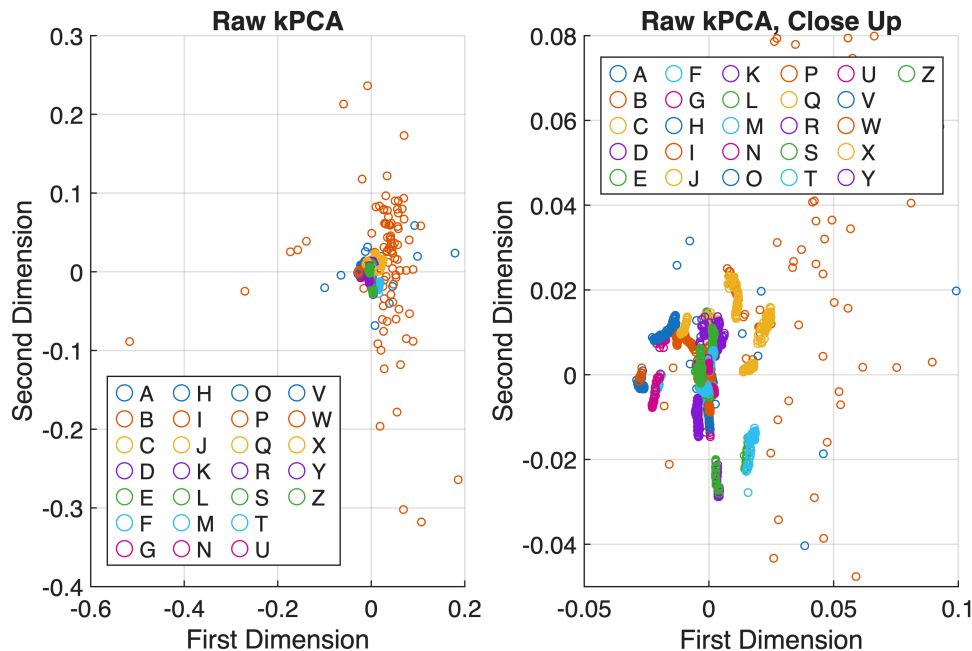
% raw
% overall
figure;
subplot(1,2,1);
hold on;
for i = 1:26
    point = row_helper(labels, class_names(i));
    scatter(reduced(point, 1), reduced(point,2), 8);
end
title('Raw kPCA');
xlabel('First Dimension');
ylabel('Second Dimension');
table = legend(cellstr(class_names), 'Location','southwest');
table.NumColumns = 4;
grid on;
hold off;
cmap = jet(10);
colormap(cmap);
% zoomed in
subplot(1,2,2);
hold on;
for i = 1:26
    point = row_helper(labels, class_names(i));
    scatter(reduced(point, 1), reduced(point,2), 8);
end
title('Raw kPCA, Close Up');

```

```

% zooming in
xlim([-0.05 0.1]);
ylim([-0.05 0.08]);
xlabel('First Dimension');
ylabel('Second Dimension');
grid on;
table = legend(cellstr(class_names), 'Location','northwest');
table.NumColumns = 6;
hold off;
cmap = jet(10);
colormap(cmap);

```



```

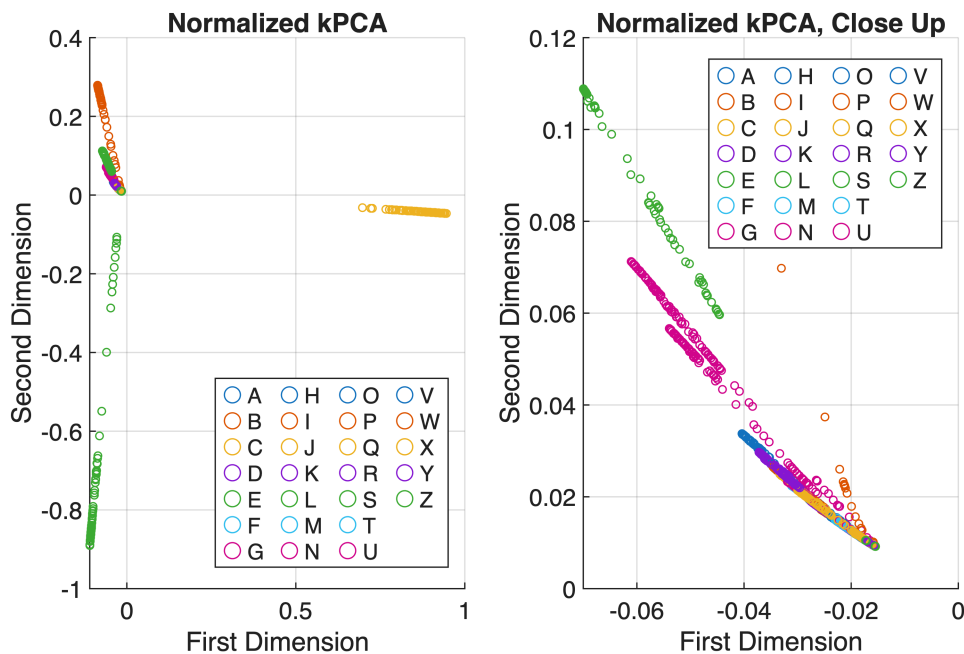
% normalized
% overall
figure;
subplot(1,2,1);
hold on;
for i = 1:26
    point = row_helper(labels, class_names(i));
    scatter(reduced_norm(point, 1), reduced_norm(point,2), 8);
end
title('Normalized kPCA');
xlabel('First Dimension');
ylabel('Second Dimension');
table = legend(cellstr(class_names), 'Location','southeast');
table.NumColumns = 4;
grid on;
hold off;
cmap = jet(10);
colormap(cmap);

```

```

% zoomed in
subplot(1,2,2);
hold on;
for i = 1:26
    point = row_helper(labels, class_names(i));
    scatter(reduced_norm(point, 1), reduced_norm(point,2), 8);
end
title('Normalized kPCA, Close Up');
% zooming in
xlim([-0.07 0]);
ylim([0 0.12]);
xlabel('First Dimension');
ylabel('Second Dimension');
grid on;
table = legend(cellstr(class_names));
table.NumColumns = 4;
hold off;
cmap = jet(10);
colormap(cmap);

```



```

images_length_kpca_normalized = size(reduced_norm, 1);
% to have same result, from sample code
rng('default');
reduced_kpca = randperm(images_length_kpca_normalized);
% recalculating with the length
% like pca and raw's split
train_percentage_kpca = .8 * images_length_kpca_normalized;
validation_percentage_kpca = .9 * images_length_kpca_normalized;
% training
reduced_kpca_training_elements = reduced_kpca(1:train_percentage_kpca);
reduced_kpca_training_labels = labels(reduced_kpca_training_elements);

```

```

reduced_kpca_x_training = reduced_norm(reduced_kpca_training_elements, :);
% validation
reduced_kpca_validation_elements =
reduced_kpca(train_percentage_kpca+1:validation_percentage_kpca);
reduced_kpca_validation_labels = labels(reduced_kpca_validation_elements);
reduced_kpca_x_validation =
reduced_norm(reduced_kpca_validation_elements, :);
% testing
reduced_kpca_testing_element =
reduced_kpca(validation_percentage_kpca+1:end);
reduced_kpca_testing_labels = labels(reduced_kpca_testing_element);
reduced_kpca_x_testing = reduced_norm(reduced_kpca_testing_element, :);
svm_helper(reduced_kpca_x_training, reduced_kpca_training_labels,
reduced_kpca_x_validation, reduced_kpca_validation_labels,
reduced_kpca_x_testing, reduced_kpca_testing_labels, 'Normalized kPCA ');

```

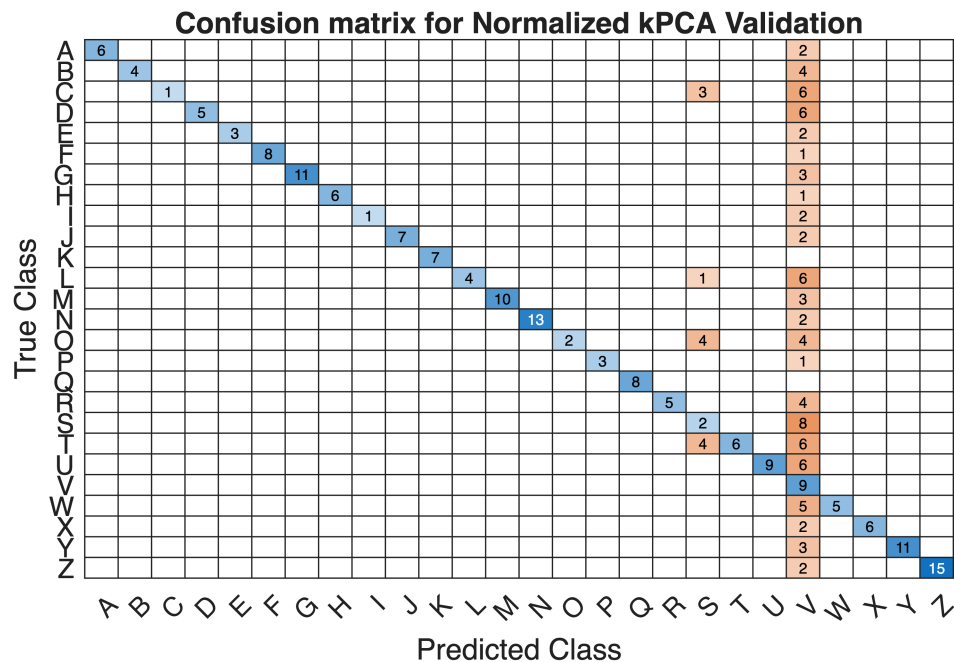
The first five class predictions with their probabilities:

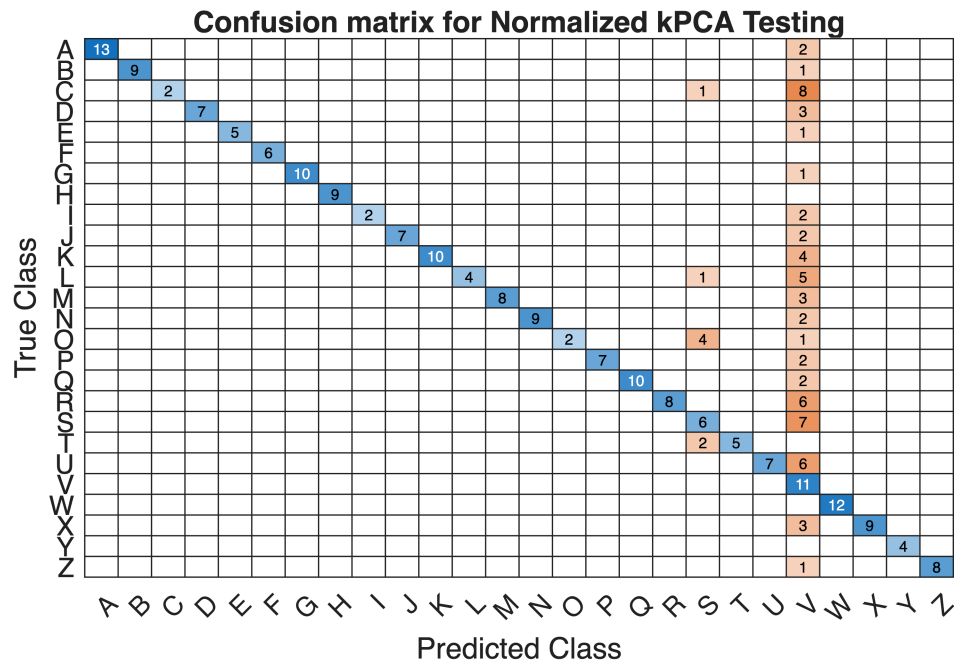
```

{'Z'}
{'H'}
{'W'}
{'N'}
{'X'}

```

-0.6023	-0.4400	-0.3234	-0.4975	-0.6609	-0.5827	-0.7607	-0.5562	-0.4058	-0.7815	-0.6023
-0.5907	-0.4335	-0.3162	-0.4869	-0.6472	-0.5679	-0.7608	0	-0.3982	-0.7788	-0.5907
-0.6050	-0.4233	-0.2968	-0.4842	-0.6233	-0.5734	-0.7834	-0.5498	-0.3787	-0.7975	-0.6050
-0.5854	-0.4377	-0.3389	-0.4868	-0.6619	-0.5713	-0.7478	-0.5443	-0.4190	-0.7745	-0.5854
-0.6237	-0.4143	-0.2400	-0.4984	-0.5987	-0.5948	-0.8358	-0.5781	-0.3275	-0.8337	-0.6237





F1 score for validation dataset is 0.708514

F1 score for testing dataset is 0.790422

Accuracy for validation dataset is 0.642308

Accuracy for testing dataset is 0.730769

```
cross_val_helper(reduced_norm, labels, 'Normalized kPCA');
```

F1 score after 3 fold cross validation is 0.775225

Accuracy after 3 fold cross validation is 0.711538

```
% one more plot with playing with parameters to achieve higher accuracy
```

```
figure;
```

```
hold on;
```

```
% tested with various nums, landed on 15 and 40 being the best
```

```
% representation
```

```
[mappedX, mapping] = kernel_pca(trainset_norm, 40, 'gauss', 15);
```

```
Computing kernel matrix...
```

```
Eigenanalysis of kernel matrix...
```

```
Computing final embedding...
```

```
for j = 1:26
```

```
    point = row_helper(labels, class_names(j));
```

```
    scatter(mappedX(point, 1), mappedX(point, 2), 8);
```

```
end
```

```
title('Adjusted Normalized kPCA');
```

```
xlabel('First Dimension');
```

```
ylabel('Second Dimension');
```

```
grid on;
```

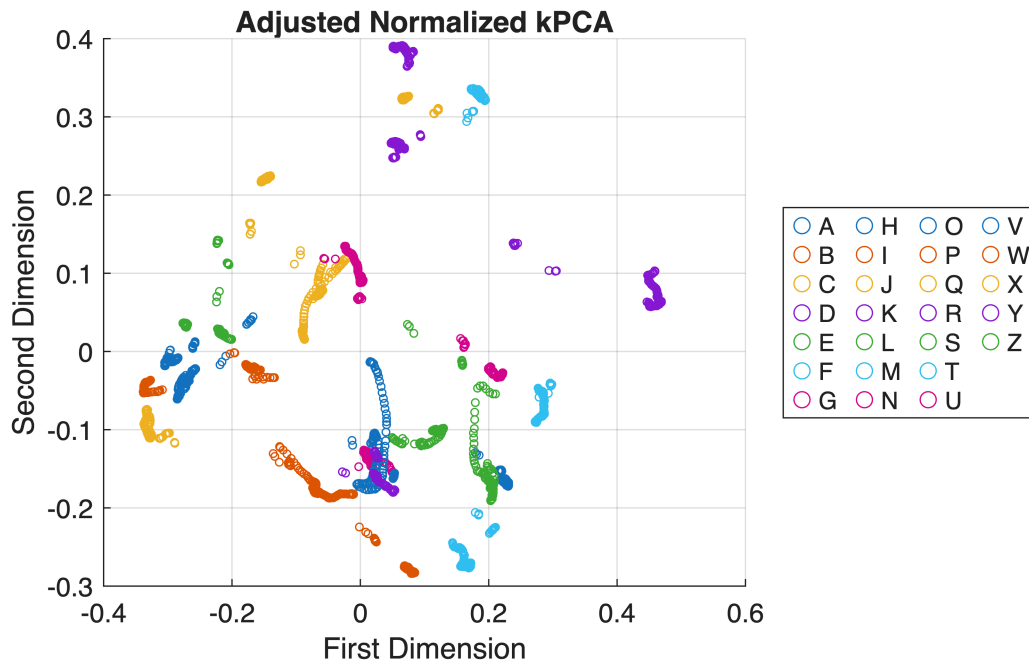
```
hold off;
```

```
cmap = jet(10);
```

```
colormap(cmap);
```

```
table = legend(cellstr(class_names), 'Location', 'eastoutside');
```

```
table.NumColumns = 4;
```



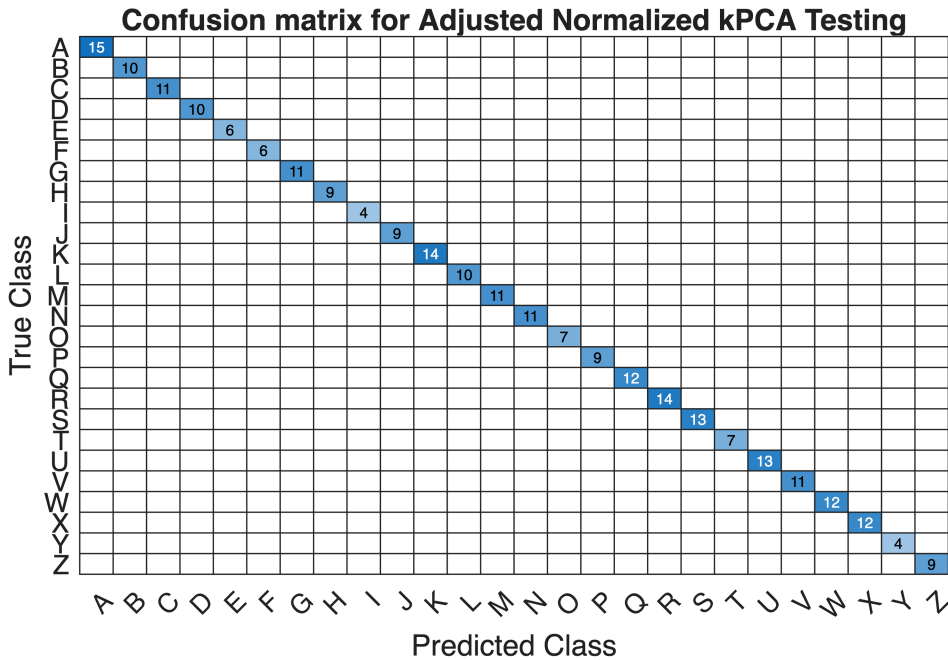
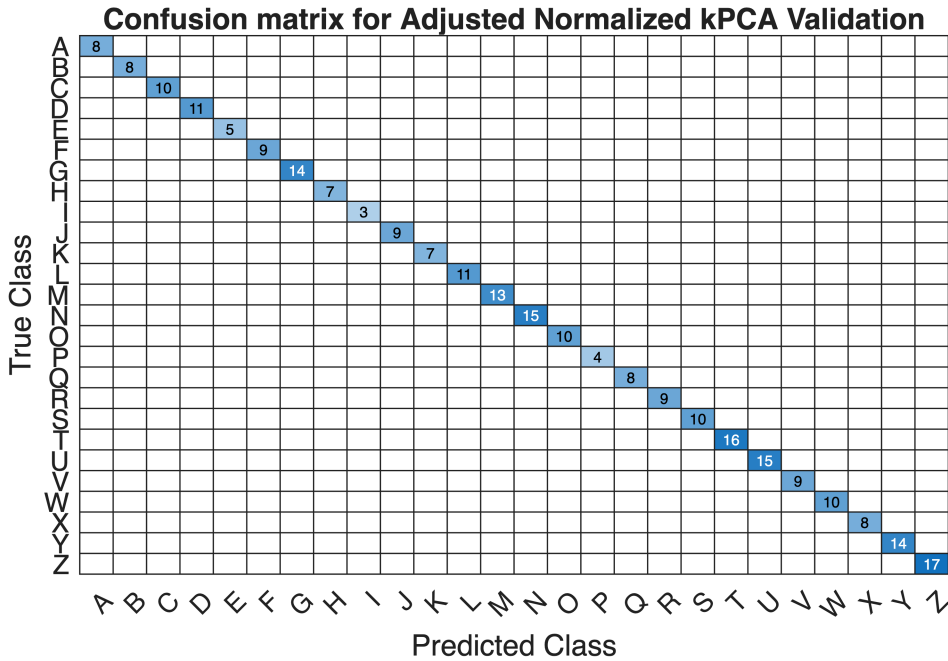
```
disp(size(mappedX, 1));
```

2600

```
images_length_kpca = size(mappedX, 1);
% to have same result, from sample code
rng('default');
% needed to redo so its okay for size issue
p_kpca = randperm(images_length_kpca);
train_percentage_kpca_adjusted = .8 * images_length_kpca;
validation_percentage_kpca_adjusted = .9 * images_length_kpca;
% same format
% training
kpca_training_elements = p_kpca(1:train_percentage_kpca_adjusted);
kpca_training_labels = labels(kpca_training_elements);
kpca_x_training = mappedX(kpca_training_elements, :);
% validation
kpca_validation_elements =
p_kpca(train_percentage_kpca_adjusted+1:validation_percentage_kpca_adjusted)
;
kpca_validation_labels = labels(kpca_validation_elements);
kpca_x_validation = mappedX(kpca_validation_elements, :);
% testing
kpca_testing_element = p_kpca(validation_percentage_kpca_adjusted+1:end);
kpca_testing_labels = labels(kpca_testing_element);
kpca_x_testing = mappedX(kpca_testing_element, :);
svm_helper(kpca_x_training, kpca_training_labels, kpca_x_validation,
kpca_validation_labels, kpca_x_testing, kpca_testing_labels, 'Adjusted
Normalized kPCA ');
```

The first five class predictions with their probabilities:

{ 'Z' }										
{ 'H' }										
{ 'W' }										
{ 'N' }										
{ 'X' }										
-0.3114	-0.3048	-0.4753	-0.5673	-0.4479	-0.7297	-0.5356	-0.6935	-0.4309	-0.6415	-0.6935
-0.5569	-0.4730	-0.4953	-0.6198	-0.5188	-0.5921	-0.5688	0	-0.5020	-0.6627	-0.6935
-0.3118	-0.3456	-0.3579	-0.6702	-0.4135	-0.6565	-0.5174	-0.6033	-0.3539	-0.6444	-0.6935
-0.6686	-0.5107	-0.5450	-0.5605	-0.6176	-0.5511	-0.4030	-0.6117	-0.4750	-0.6208	-0.6935
-0.2476	-0.2428	-0.4191	-0.7430	-0.3923	-0.7459	-0.5034	-0.7247	-0.3333	-0.6819	-0.6935



F1 score for validation dataset is 1.000000
F1 score for testing dataset is 1.000000
Accuracy for validation dataset is 1.000000
Accuracy for testing dataset is 1.000000

```
cross_val_helper(mappedX,labels, 'Adjusted Normalized kPCA ');
```

F1 score after 3 fold cross validation is 1.000000

Accuracy after 3 fold cross validation is 1.000000

For Isomap, I made a helper to help plot the images so I can achieve a similar example of head pose graphs we examined in class.

```
function image_plot_helper(normalized_reduced, labels, trainset, names)
    size_names = length(names);
    % so it works for both 26 and 6 letters
    for i =1: size_names
        point = row_helper(labels, names(i));
        first_sample = find(point, 1);
        a = normalized_reduced(first_sample, 1);
        % disp(a);
        % debugging
        b = normalized_reduced(first_sample, 2);
        % disp(b);
        row = trainset(first_sample,:);
        reshaped = reshape(row, [32 32 3]);
        % so the images are not upside down and visible
        adjusted = flipud(uint8(reshaped));
        imagesc([a-0.5 a+0.5], [b-0.5 b+0.5],adjusted);
    end
end
```

Isomap

```
% Very similar format to kPCA
i_trainset = [];
i_trainset_norm = [];
i_labels = strings(0,1);
t4s = tic;
for i = 1:26
    point = find(row_helper(classes, class_names(i)));
    point= point(1:100);
    i_trainset = [i_trainset; data(point,:)];
    i_trainset_norm =[i_trainset_norm; normalized(point,:)];
    i_labels = [i_labels; repmat(class_names(i),100, 1)];
    % so its remained as letters
end
% using more dim to get higher acc, f1, better cm
[i_reduced_norm, i_mapping_norm] = compute_mapping(i_trainset_norm,
'Isomap', 2, 300);
```

Constructing neighborhood graph...

Computing shortest paths...

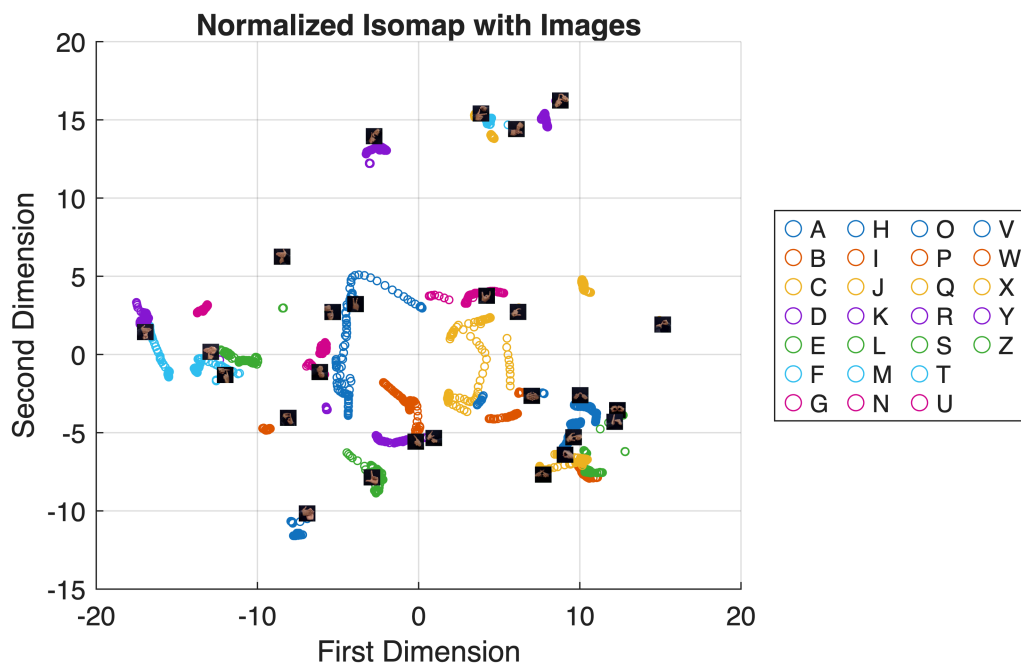
Constructing low-dimensional embedding...

```
t4 = toc(t4s);
```

```
fprintf('Isomap took %f seconds', t4);
```

Isomap took 114.203247 seconds

```
figure;
hold on;
for i = 1:26
    point = row_helper(i_labels, class_names(i));
    scatter(i_reduced_norm(point, 1), i_reduced_norm(point,2), 8);
end
title('Normalized Isomap with Images');
xlabel('First Dimension');
ylabel('Second Dimension');
grid on;
table = legend(cellstr(class_names), 'Location','eastoutside');
table.NumColumns = 4;
image_plot_helper(i_reduced_norm, i_labels, i_trainset, class_names);
cmap = jet(10);
colormap(cmap);
hold off;
```

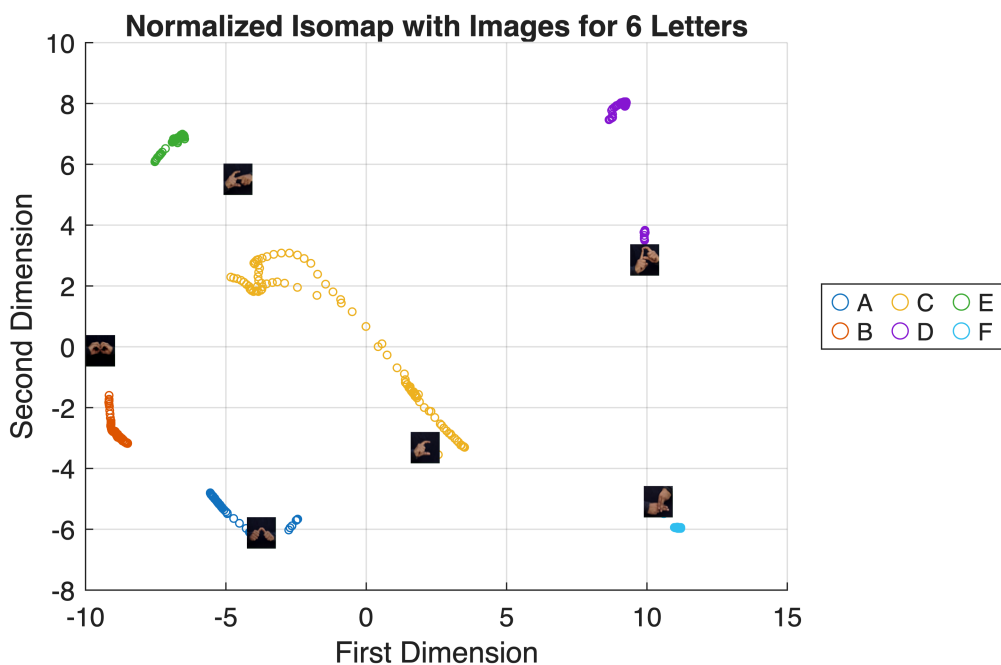


```
% one more plot to visualize the 6 letters at the beginning
l_trainset = [];
l_trainset_norm = [];
l_labels = strings(0,1);
for i = 1:6
    point = find(classes == letters(i));
    point= point(1:100);
    l_trainset = [l_trainset; data(point,:)];
    l_trainset_norm = [l_trainset_norm; normalized(point,:)];
    l_labels = [l_labels; repmat(letters(i),100, 1)];
    % so its remained as letters
```

```
end
[l_reduced_norm, l_mapping_norm] = compute_mapping(l_trainset_norm,
'Isomap', 2, 300);
```

Constructing neighborhood graph...
Computing shortest paths...
Constructing low-dimensional embedding...

```
figure;
hold on;
for i = 1:6
    point = (l_labels == letters(i));
    scatter(l_reduced_norm(point, 1), l_reduced_norm(point,2), 8);
end
title('Normalized Isomap with Images for 6 Letters');
xlabel('First Dimension');
ylabel('Second Dimension');
grid on;
% so the images are not upside down
table = legend(cellstr(letters), 'Location','eastoutside');
table.NumColumns = 4;
image_plot_helper(l_reduced_norm, l_labels, l_trainset, letters);
cmap = jet(10);
colormap(cmap);
hold off;
```



```
isomap_images_length = size(i_reduced_norm, 1);
disp(isomap_images_length);
```

2600

```
% to have same result, from sample code
rng('default');
```

```

p_isomap = randperm(isomap_images_length);
tr_percentage_isomap = .8 * isomap_images_length;
val_percentage_isomap = .9 * isomap_images_length;
% training
i_training_elements = p_isomap(1:tr_percentage_isomap);
i_training_labels = i_labels(i_training_elements);
i_x_training = i_reduced_norm(i_training_elements, :);
% testing
i_testing_element = p_isomap(val_percentage_isomap+1:end);
i_testing_labels = i_labels(i_testing_element);
i_x_testing = i_reduced_norm(i_testing_element, :);
% validation
i_validation_elements =
p_isomap(tr_percentage_isomap+1:val_percentage_isomap);
i_validation_labels = i_labels(i_validation_elements);
i_x_validation = i_reduced_norm(i_validation_elements, :);
% calling helpers
svm_helper(i_x_training, i_training_labels, i_x_validation,
i_validation_labels, i_x_testing, i_testing_labels, 'Normalized Isomap ');

```

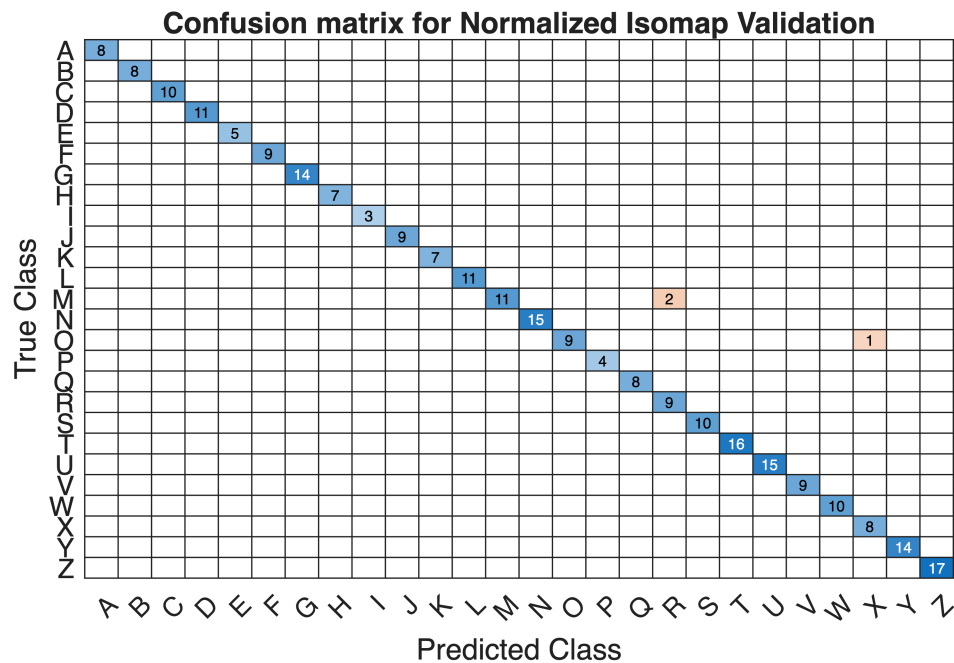
The first five class predictions with their probabilities:

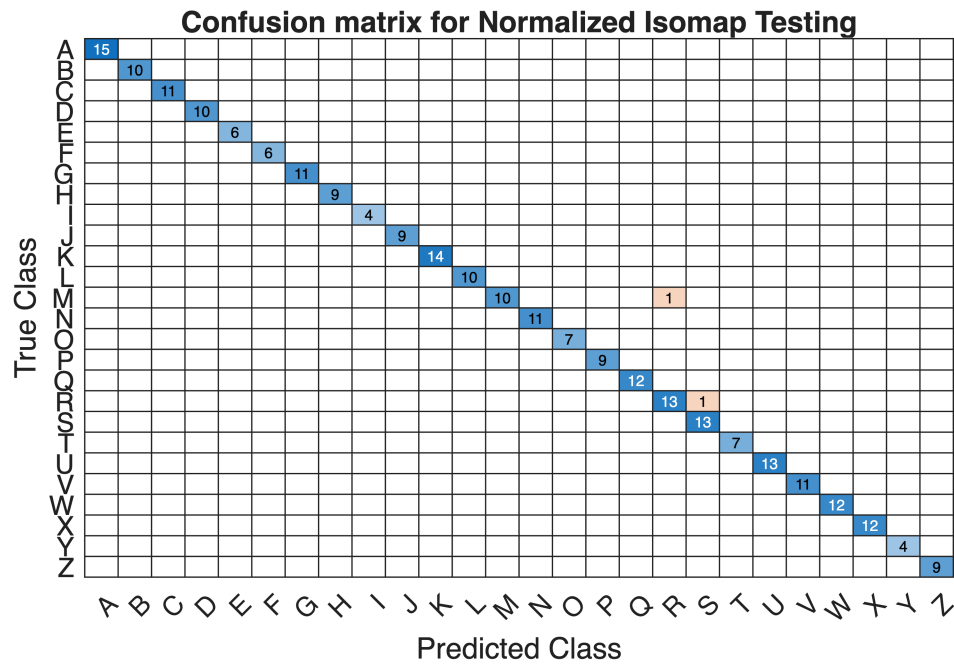
```

{'Z'}
{'H'}
{'W'}
{'N'}
{'X'}

```

-0.4995	-0.1030	-0.2882	-0.6246	-0.5567	-0.6436	-0.5939	-0.6261	-0.4984	-0.6317	-0.6317
-0.5117	-0.5428	-0.2618	-0.6120	-0.5513	-0.6315	-0.5800	0	-0.4807	-0.6195	-0.6195
-0.5019	-0.5389	-0.2518	-0.6081	-0.5474	-0.6277	-0.5761	-0.6096	-0.4769	-0.6157	-0.6157
-0.5122	-0.5431	-0.2627	-0.6121	-0.5516	-0.6316	-0.5802	-0.6136	-0.4812	-0.6196	-0.6196
-0.5425	-0.1689	-0.3111	-0.6360	-0.3688	-0.6546	-0.6063	-0.6375	-0.5140	-0.6427	-0.6427





F1 score for validation dataset is 0.988662
 F1 score for testing dataset is 0.993997
 Accuracy for validation dataset is 0.988462
 Accuracy for testing dataset is 0.992308

```
cross_val_helper(i_reduced_norm,i_labels, 'Normalized Isomap');
```

F1 score after 3 fold cross validation is 0.990768
 Accuracy after 3 fold cross validation is 0.990769